
UNIT 11 DATABASE RECOVERY AND SECURITY

- 11.0 Introduction
- 11.1 Objectives
- 11.2 What Is Recovery?
 - 11.2.1 Kinds of Failures
 - 11.2.2 Storage Structures for Recovery
 - 11.2.3 Recovery and Atomicity
 - 11.2.4 Transactions and Recovery
 - 11.2.5 Recovery in Small Databases
- 11.3 Transaction Recovery
 - 11.3.1 Log-Based Recovery
 - 11.3.2 Checkpoints in Recovery
 - 11.3.3 Recovery Algorithms
 - 11.3.4 Recovery with Concurrent Transactions
 - 11.3.5 Buffer Management
 - 11.3.6 Remote Backup Systems
- 11.4 Security in Commercial Databases
 - 11.4.1 Common Database Security Failures
 - 11.4.2 Database Security Levels
 - 11.4.3 Relationship Between Security and Integrity
 - 11.4.4 Difference Between Operating System And Database Security
- 11.5 Access Control
 - 11.5.1 Authorisation of Data Items
 - 11.5.2 A Basic Model of Database Access Control
 - 11.5.3 SQL Support for Security and Recovery
- 11.6 Audit Trails in Databases
- 11.7 Summary
- 11.8 Solutions/Answers

11.0 INTRODUCTION

In the previous unit of this block, you have gone through the details of transactions, their properties, and the management of concurrent transactions. In this unit, you will be introduced to two important issues relating to database management systems – how to deal with database failures and how to handle database security. A computer system suffers from different types of failures. A DBMS controls very critical data of an organisation and, therefore, must be reliable. However, the reliability of the database system is also linked to the reliability of the computer system on which it runs. The types of failures that the computer system is likely to be subjected to include failures of components or subsystems, software failures, power outages, accidents, unforeseen situations and natural or man-made disasters. Database recovery techniques are methods of making the database consistent till the last possible consistent state. Thus, the basic objective of the recovery system is to resume the database system to the point of failure with almost no loss of information. Further, the recovery cost should be justifiable. In this unit, we will discuss various types of failures and some of the approaches to database recovery.

The second main issue that is being discussed in this unit is Database security. “Database security” is the protection of the information contained in the database against unauthorised access, modification, or destruction. The first condition for security is to have Database integrity. “Database integrity” is the mechanism that is applied to ensure that the data in the database is consistent. In addition, the unit discusses various access

control mechanisms for database access. Finally, the unit introduces the use of audit trails in a database system.

11.1 OBJECTIVES

At the end of this unit, you should be able to:

- describe the terms recovery;
- explain log based recovery;
- explain the use of checkpoints in recovery;
- define the various levels of database security;
- define the access control mechanism;
- identify the use of audit trails in database security.

11.2 WHAT IS RECOVERY?

During the life of a transaction, i.e. after the start of a transaction but before the transaction commits, it makes several uncommitted changes in data items of a database. The database during this time may be in an inconsistent state. In practice, several things might happen to prevent a transaction from completing. Recovery techniques are designed to bring an inconsistent database, after a failure, into a consistent database state. If a transaction completes normally and commits, then all the changes made by that transaction on the database should get permanently registered in the database. They should not be lost (please recollect the durability property of transactions given in Unit 10). But if a transaction does not complete normally and terminates abnormally then all the changes made by it should be discarded.

11.2.1 Kinds of Failures

An abnormal termination of a transaction may occur due to several reasons, including:

- a) user may decide to abort the transaction issued by him/her,
- b) there might be a deadlock in the system,
- c) there might be a system failure.

The recovery mechanisms must ensure that a consistent state of the database can be restored under all circumstances. In case of transaction abort or deadlock, the system remains in control and can deal with the failure, but in case of a system failure, the system loses control because the computer itself has failed. Will the results of such failure be catastrophic? A database contains a huge amount of useful information, and any system failure should be recognised on the system restart. The DBMS should recover from any such failures. Let us first discuss the kinds of failure for ascertaining the approach of recovery.

A DBMS may encounter a failure. These failures may be of the following types:

1. Transaction failure: An ongoing transaction may fail due to:

- **Logical errors:** Transaction cannot be completed due to some internal error condition.
- **Database system errors:** A database system error can be caused by some failure at the level of a database. For example, a transaction deadlock may result in a database system error. This will result in the abrupt termination of some of the ongoing deadlocked transactions.

2. System crash: This kind of failure includes hardware/software failure of a computer system. In addition, a sudden power failure can also result in a system crash, which may result in the following:

- Loss or corruption of non-volatile storage contents.
- Loss of contents of the entire disk or parts of the disk. However, such loss is assumed to be detectable; for example, the checksums used on disk drives can detect this failure.

11.2.2 Storage Structures for Recovery

All these failures result in the inconsistent state of a database. Thus, we need a recovery scheme in a database system, but before we discuss recovery, let us briefly define the storage structure from the recovery point of view.

There are various ways for storing information for database system recovery. These are:

Volatile storage: Volatile storage does not survive system crashes. Examples of volatile storage are - the main memory or cache memory of a database server.

Non-volatile storage: The non-volatile storage survives the system crashes if it does not involve disk failure. Examples of non-volatile storage are - magnetic disk, magnetic tape, flash memory, and non-volatile (battery-backed) RAM.

Stable storage: This is a mythical form of storage structure that is assumed to survive all failures. This storage structure is assumed to maintain multiple copies on distinct non-volatile media, which may be independent disks. Further, data loss in case of disaster can be protected by keeping multiple copies of data at remote sites. In practice, software failures are more common than hardware failures. Fortunately, recovery from software failures is much quicker.

11.2.3 Recovery and Atomicity

The concept of recovery relates to the atomic nature of a transaction. Atomicity is the property of a transaction, which states that a transaction is a complete unit. Thus, the execution of a part transaction can lead to an inconsistent state of the database, which may require database recovery. Let us explain this with the help of an example:

Assume that a transaction transfers Rs.2000/- from A's account to B's account. For simplicity, we are not showing any error checking in the transaction. The transaction may be written as:

Transaction T1:

```
READ A
A = A - 2000
WRITE A
—————> Failure
READ B
B = B + 2000
WRITE B
COMMIT
```

What would happen if the transaction failed after account A has been written back to the database? As far as the holder of account A is concerned s/he has transferred the money but that has never been received by account holder B.

Why did this problem occur? Because although a transaction is atomic, yet it has a life cycle during which the database gets into an inconsistent state and failure has occurred at that stage.

What is the solution? In this case, where the transaction has not yet committed the

changes made by it, the partial updates need to be undone.

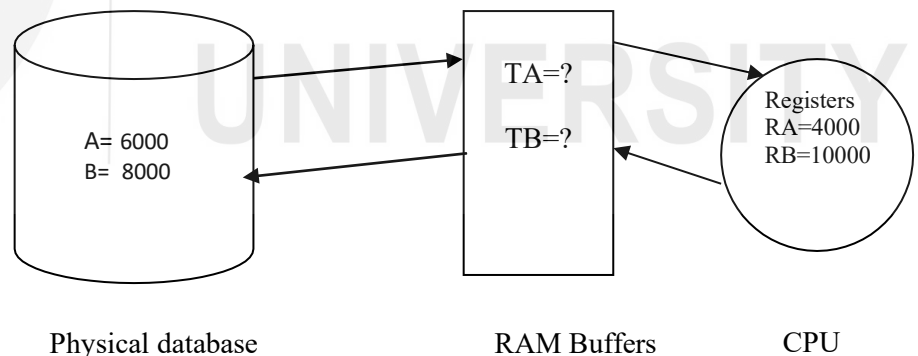
How can we do that? By remembering information about a transaction such as - when did it start, what items it updated etc. All such details are kept in a log file. We will study log files later in this unit when we define different methods of recovery. In the next section, we discuss the recovery of transactions in more detail.

11.2.4 Transactions and Recovery

The basic unit of recovery is a transaction. But how are the transactions handled during recovery?

Consider the following two cases:

- i. Consider that some transactions are deadlocked, then at least one of these transactions must be restarted to break the deadlock, and thus, the partial updates made by this restarted transaction program are to be undone to keep the database in a consistent state. In other words, you may **ROLLBACK** the effect of a transaction.
- ii. A transaction has committed, but the changes made by the transaction have not been communicated to the physical database on the hard disk. A software failure now occurs, and the contents of the CPU/ RAM are lost. This leaves the database in an inconsistent state. Such failure requires that on restarting the system the database be brought to a consistent state using **redo** operation. The redo operation performs the changes made by the transaction again to bring the system to a consistent state. The database system can then be made available to the users. The point to be noted here is that such a situation has occurred as database updates are performed in the buffer memory. *Figure 1* shows cases of **undo** and **redo**.



	Physical Database at the time of failure	RAM	Activity
Case 1	A=6000 B=8000	TA=4000 TB=8000	Transaction T1 has changed the value of A in register RA (4000) and RB (10000). RA is written to RAM (TA), but not updated in A. RB is not written back to RAM (TB). T1 did not COMMIT. Now, T1 is aborted by the user. The value of CPU registers and RAM

			are made irrelevant. You cannot determine if the TA, TB has been written back to A, B. You must UNDO the transaction.
Case 2	A=4000 B=8000	TA=4000 TB=8000	The value of A in the physical database has got updated due to buffer management. RB is not written back to RAM (TB). The transaction did not COMMIT so far. Now, the transaction T1 aborts. You must UNDO the transaction.
Case 3	A=6000 B=8000	TA=4000 TB=10000 COMMIT	The value B in the physical database has not got updated due to buffer management. T1 has raised the COMMIT flag. The changes of the transaction must be performed again. You must REDO the transaction. How? (Discussed in later sections).

Figure 1: Database Updates and Recovery

11.2.5 Recovery in Small Databases

Failures can be handled using different recovery techniques that are discussed later in the unit. But the first question is: Do you really need recovery techniques as a failure control mechanism? The recovery techniques are somewhat expensive both in terms of time and memory space for small systems. In such a case, it is beneficial to avoid failures by some checks instead of deploying recovery techniques to make the database consistent. Also, recovery from failure involves manpower that can be used in other productive work if failures can be avoided. It is, therefore, important to find some general precautions that help control failures. Some of these precautions may be:

- to regulate the power supply.
- to use a failsafe secondary storage system such as RAID.
- to take periodic database backups and keep track of transactions after each recorded state.
- to properly test transaction programs prior to use.
- to set important integrity checks in the databases as well as user interfaces.

However, it may be noted that if the database system is critical to an organisation, it must use a DBMS that is suitably equipped with recovery procedures.

11.3 TRANSACTION RECOVERY

As discussed in the previous section, a transaction is the unit of recovery. A commercial database system, like a banking system, may support many concurrent transactions at a time. A failure may affect multiple transactions in such systems. Several recovery techniques have been designed for commercial DBMSs. This section discusses some of the basic recovery schemes used in commercial DBMSs.

11.3.1 Log-Based Recovery

Let us first define the term transaction log in the context of DBMS. A transaction log, in DBMS, records information about every transaction that modifies any data values in the database. A log contains the following information about a transaction:

- A transaction BEGIN marker.
- Transaction identification - transaction ID, terminal ID, user ID, etc.
- The operations being performed by the transaction such as UPDATE, DELETE, INSERT.
- The data items or objects affected by the transaction - may include the table's name, row number and column number.
- Before or previous values (also called UNDO values) and after or changed values (also called REDO values) of the data items that have been updated.
- A pointer to the next transaction log record, if needed.
- The COMMIT marker of the transaction.

In a database system, several transactions run concurrently. When a transaction commits, the data buffers used by it need not be written back to the physical database stored on the secondary storage as these buffers may be used by several other transactions that have not yet committed. On the other hand, some of the data buffers that may have been updated by several uncommitted transactions might be forced back to the physical database, as they are no longer being used by the database. So, the transaction log helps in remembering which transaction did what changes. Thus, the system knows exactly how to separate the changes made by transactions that have already committed from those changes that are made by the transactions that did not yet COMMIT. Any operation such as BEGIN transaction, INSERT/ DELETE/ UPDATE and transaction COMMIT, adds information to the log containing the transaction identifier and enough information to **UNDO** or **REDO** the changes.

But how do we recover using a log? Let us demonstrate this with the help of an example having three concurrent transactions that are active on ACCOUNTS relation:

Transaction T1	Transaction T2	Transaction T3
READ X	READ A	READ Z
SUBTRACT 100	ADD 200	SUBTRACT 500
WRITE X	WRITE A	WRITE Z
READ Y		
ADD 100		
WRITE Y		

Figure 2: The sample transactions

Assume that these transactions have the following log file (hypothetical structure):

Transaction Begin Marker	Transaction Id	Operation on ACCOUNTS table	UNDO values (assumed)	REDO values	Transaction Commit Marker
Yes	T1	SUB ON X ADD ON Y	X=500 Y=800	X=400 not done yet	No
Yes	T2	ADD ON A	A=1000	A=1200	No
Yes	T3	SUB ON Z	Z=900	Z=400	Yes

Figure 3: A sample (hypothetical) Transaction log

Now assume at this point of time a failure occurs, then how the recovery of the database will be done on restart.

Transaction /Values	Initial (UNDO value)	Just before the failure	Operation Required for recovery	Database Values after Recovery
T1/X	500	400 (assuming update has been done in physical database also)	UNDO (As transaction did not COMMIT)	X = 500
T1/Y	800	800	UNDO	Y = 800

T2/A	1000	1000 (assuming update has not been done in physical database)	UNDO (As transaction did not COMMIT)	A = 1000
T3/Z	900	900 (assuming update has not been done in physical database)	REDO (As transaction <i>did</i> COMMIT)	Z = 400

Figure 4: The database recovery

The selection of REDO or UNDO for a transaction for the recovery is done based on the state of the transactions. This state is determined in two steps:

- Look into the log file and find all the transactions that have started. For example, in *Figure 3*, transactions T1, T2 and T3 are candidates for recovery.
- Find those transactions that have COMMITTED. REDO these transactions. All other transactions have not COMMITTED, so they should be rolled back, so UNDO them. For example, in *Figure 3*, UNDO will be performed on T1 and T2, and REDO will be performed on T3.

Please note that in *Figure 4*, some of the values may not have yet been communicated to the database, yet we need to perform UNDO as we are not sure what values have been written back to the database. Similarly, you must perform REDO operations on committed transactions, such as Transaction T3 in *Figure 3* and *Figure 4*.

But how will the system recover? Once the recovery operation has been specified, the system just determines the required REDO or UNDO values from the transaction log and changes the inconsistent state of the database to a consistent state. (Please refer to *Figure 3* and *Figure 4*).

13.3.2 Checkpoints in Recovery

Let us consider several transactions, which are shown on a timeline, with their respective START and COMMIT time (see *Figure 5*).

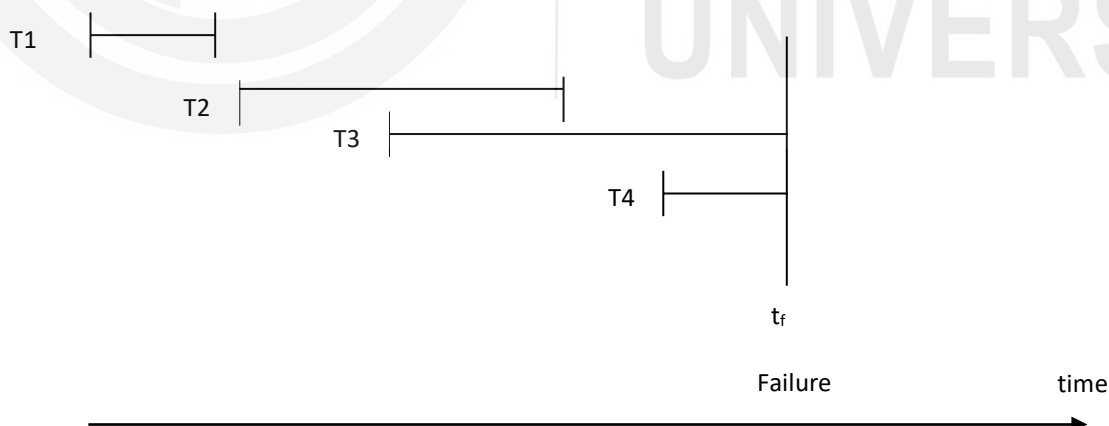


Figure 5: Execution of Concurrent Transactions

Consider that the transactions are allowed to execute concurrently, on encountering a failure at time t_f , the transactions T1 and T2 are to be REDONE and T3 and T4 will be UNDONE (Refer to *Figure 5*). Now, considering that a system allows thousands of parallel transactions, then all those transactions that have been committed may have to be redone, and all uncommitted transactions need to be undone. That is not a very good

choice as it requires redoing even those transactions that might have been committed several hours earlier. How can you improve this situation? You can remedy this problem by taking a checkpoint. *Figure 6* shows a checkpoint mechanism:

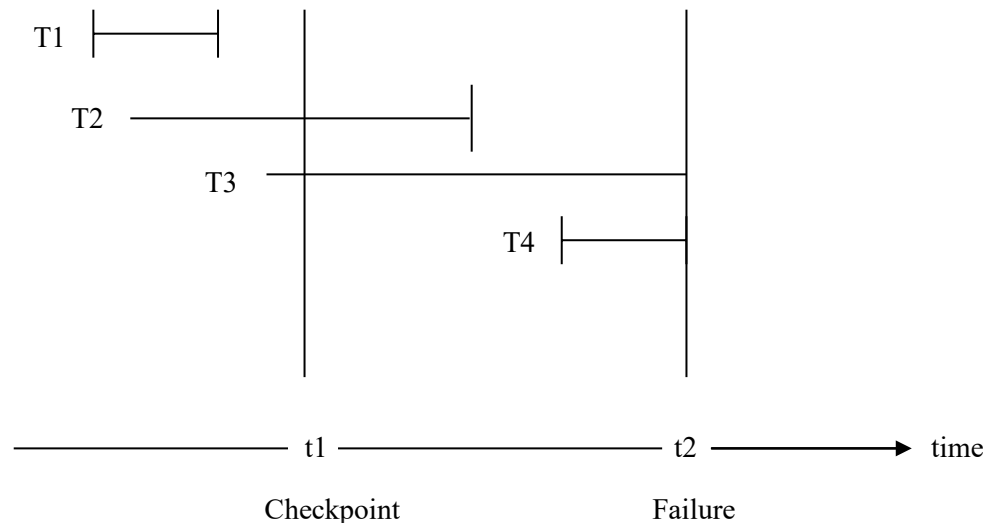


Figure 6: Checkpoint in Transaction Execution

A checkpoint is taken at time t1, and a failure occurs at time t2. Checkpoint transfers all the committed changes to the database and all the system logs to stable storage (the storage that would not be lost). On the restart of the system after the failure, the stable checkpointed state is restored. Thus, we need to REDO or UNDO only those transactions that have been completed or started after the checkpoint has been taken. A disadvantage of this scheme is that the database would not be available when the checkpoint is being taken. In addition, some of the uncommitted data values may be put in the physical database. To overcome the first problem, the checkpoints should be taken at times when the system load is low. To avoid the second problem, the system may allow the ongoing transactions to be complete while not starting any new transactions.

In the case of *Figure 6*, the recovery from failure at time t2 will be as follows:

- The transaction T1 will not be considered for recovery, as the changes made by it have already been committed and transferred to the physical database at checkpoint t1.
- The transaction T2 has not committed till checkpoint t1 but has committed before t2 will be REDONE.
- T3 must be UNDONE as the changes made by it before the checkpoint (we do not know for sure if any such changes were made prior to the checkpoint) must have been communicated to the physical database. T3 must be restarted with a new name.
- T4 started after the checkpoint, and if we strictly follow the scheme in which the buffers are written back only on the checkpoint, then nothing needs to be done except restart the transaction T4 with a new name.

The restart of a transaction requires the log to keep information on the new name of the transaction. This new transaction may be given higher priority.

But one question that remains unanswered is - during a failure, we lose database information in RAM buffers; we may also lose the content of the log as it is also stored in RAM buffers, so how does the log ensure recovery?

The answer to this question lies in the fact that for storing the log of the transaction, we follow a **Write Ahead Log Protocol**. As per this protocol, the transaction logs are written to stable storage as follows:

- **UNDO portion of the log is written to stable storage prior to any updates. and**
- **REDO portion of the log is written to stable storage prior to the commit.**

Log-based recovery scheme can be used for any kind of failure provided you have stored the most recent checkpoint state and most recent log as per write-ahead log protocol into the stable storage. Stable storage from the viewpoint of external failure requires more than one copy of such data at more than one location. You can refer to further readings for more details on recovery and its techniques.

☞ Check Your Progress 1

1) What is the need for recovery? What is the basic unit of recovery?

.....

.....

2) What is the log-based recovery?

.....

.....

3) What is a checkpoint? Why is it needed? How does a checkpoint help in recovery?

.....

.....

11.3.3 Recovery Algorithms

As discussed earlier, a database failure may bring the database into an inconsistent state, as many ongoing transactions will simply abort. In order to bring such an inconsistent database to a consistent state, you are required to use recovery algorithms. Database recovery algorithms require basic data related to failed transactions. Thus, a recovery algorithm requires the following actions:

- 1) Actions are taken to collect the required information for recovery while the transactions are being processed prior to the failure.
- 2) Actions are taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability.

In the context of point 1 above, it may be noted that the information about the changes made by a transaction is recorded in a log file. In general, the sequence of logging during the transaction execution is as follows:

- A transaction, say T_i , announces its start by putting a log record consisting of $\langle T_i \text{ start} \rangle$.
- Before T_i executes the **write**(X) (see Figure 7), put a log record $\langle T_i, X, V_1, V_2 \rangle$, where V_1 represented the older value of X , i.e., the value of X before the write (*undo* value), and V_2 is the updated value of X (*redo* value).
- On successful completion of the last statement of the transaction T_i , a log

record consisting of $\langle T_i \text{ commit} \rangle$ is put in the log. It may be noted that all these log records are put in stable storage media, that is, they are not buffered in the main memory).

Two approaches for recovery using logs are:

- Deferred database modification.
- Immediate database modification.

Deferred Database Modification

This scheme logs all the changes made by a transaction into a log file, which is stored on stable storage. Further, these changes are not made in the database till the transaction commits. Let us assume that transactions execute serially to simplify the discussion. Consider a transaction T_i , it performs the following sequence of log file actions:

Event	Record for Log file	Comments
Start of Transaction	$\langle T_i \text{ start} \rangle$	
write (X)	$\langle T_i, X, V \rangle$	Transaction writes a new value V for data item X in the database.
Transaction T_i commits	$\langle T_i \text{ commit} \rangle$	The transaction has been committed, and now the deferred updates can be written to the database

- Please note that in this approach, the old values of the data items are not saved at all, as changes are being recorded in the log file and the database is not changed till a transaction commits. For example, the **write**(X), shown above, does not write the value of X in the database till the transaction commits. However, the record $\langle T_i, X, V \rangle$ is written to the log. All these updates are written to the database after the transaction commits.

How is the recovery performed for the deferred database modification scheme?

For the database recovery after the failure, the log file is checked. The transactions for which the log file contains the $\langle T_i \text{ start} \rangle$ and the $\langle T_i \text{ commit} \rangle$ records, the REDO operation is performed using log record $\langle T_i, X, V \rangle$. Why? Because in the deferred update scheme we do not know if the changes made by this committed transaction have been carried out in the physical database or not. The Redo operation can be performed many times without the loss of information, so it will be applicable even if the crash occurs during recovery. The transactions for which $\langle T_i \text{ start} \rangle$ is in the log file but not the $\langle T_i \text{ commit} \rangle$, the transaction UNDO is not required, as it is expected that the values are not yet written to the database.

Let us explain this scheme with the help of the transactions given in Figure 7.

T_1 : READ (X) $X = X - 1000$ WRITE (X) READ (Y) $Y = Y + 1000$ WRITE (Y)	T_2 : READ (Z) $Z = Z - 1000$ WRITE (Z)
--	---

Sample Transactions T_1 and T_2 (T_1 executes before T_2)

Figure 7: Sample Transaction for demonstrating Recovery Algorithm

Figure 8 shows the state of a sample log file for three possible failure instances, namely

(a), (b) and (c). (Assuming that the initial balance in X is 10,000/- Y is 5,000/- and Z has 20,000/-):

<T ₁ START>	<T ₁ START >	<T ₁ START>
<T ₁ , X, 9000>	<T ₁ , X, 9000>	<T ₁ , X, 9000>
<T ₁ , Y, 6000>	<T ₁ , Y, 6000>	<T ₁ , Y, 6000>
	<T ₁ COMMIT>	<T ₁ COMMIT >
	<T ₂ START >	<T ₂ START >
	<T ₂ , Z, 19000>	<T ₂ , Z, 19000>
		<T ₂ COMMIT >
(a)	(b)	(c)

Figure 8: Log Records for Deferred Database Modification for Transactions of Figure 7

Why do you store only the Redo value in this scheme? The reason is that No UNDO is required as updates are communicated to stable storage only after COMMIT or even later. The following REDO operations would be required if the log on stable storage at the time of the crash is as shown in Figure 8(a) 8(b) and 8(c).

- (a) No REDO is needed, as no transaction has been committed in Figure 8(a).
- (b) Perform REDO(T_1), as <T₁ COMMIT> is part of the log in Figure 8 (b).
- (c) Perform REDO(T_1) and REDO(T_2), as <T₁ COMMIT > and <T₂ COMMIT > are in the log shown in Figure 8(c).

Please note that you can repeat this sequence of redo operation as suggested in Figure 8(c) any number of times, it will still bring the value of X, Y, and Z to consistent redo values. This property of the redo operation is called **idempotent**.

Immediate Database Modification

This recovery scheme allows the modification of the data items of the stored database by ongoing uncommitted transactions. Thus, the recovery database would require UNDO and REDO of updates made by these transactions. Therefore, the log file for this recovery scheme should include the UNDO value or the original value and the REDO value or the modified value. Further, the log records are assumed to be written to the stable storage.

The following log file shows the log of transactions given in Figure 7, for the case when immediate database modification is followed:

Log	Write operation	Output
<T ₁ START>		
<T ₁ , X, 10000, 9000>	X = 9000	Output Block of X
<T ₁ , Y, 5000, 6000>	Y = 6000	Output Block of Y
<T ₁ COMMIT>		
<T ₂ START>		
<T ₂ , Z, 20,000, 19,000>	Z = 19000	Output Block of Z
<T ₂ COMMIT>		

Figure 9: Log Records for Immediate Database Modification for Transactions of Figure 7

The UNDO and REDO operations for this recovery scheme are as follows:

- **UNDO(T_i):**
 - List all the transactions which have the start record in the log but no commit record. For all the uncommitted transactions in the list perform the following:
 - Find the last UNDO log record of a transaction.
 - Move backwards in the log file to find all the entries for UNDO of

- that transaction.
 - For each UNDO entry, assign the UNDO value or original value to the data item value.
 - **REDO(T_i):**
 - List all the transactions which have the start record and commit record in the log. For all the committed transactions in the list, perform the following:
 - Find the first REDO log record of a transaction.
 - Move forward in the log file to find all the entries for REDO of that transaction.
 - For each REDO entry, assign the REDO value or modified value to the data item value.

You may note that you may perform REDO/UNDO operations any number of times if needed. Further, UNDO operations are performed first, followed by the REDO operations.

Example:

Consider the log as it appears at three instances of time.

<T₁ start> <T₁, X 10000, 9000> <T₁, Y 5000, 6000> Failure (a)	<T₁ start> <T₁, X 10000, 9000> <T₁, Y 5000, 6000> <T₁, Commit> <T₂ start> <T₂, Z 20000, 19000> Failure (b)	<T₁ start> <T₁, X 10000, 9000> <T₁, Y 5000, 6000> <T₁, Commit> <T₂ start> <T₂, Z 20000, 19000> <T₂, Commit> Failure (c)
---	--	---

Figure 10: Recovery in Immediate Database Modification technique

For each of the failures as shown in Figure 10 (a), (b) and (c), the following recovery actions would be needed:

- (a) UNDO (T_1):
 - X ← Undo value of X, i.e. 10000;
 - Y ← Undo value of Y, i.e. 5000.
- (b) UNDO (T_2):
 - Z ← Undo value of Z, i.e. 20000;
 REDO (T_1):
 - X ← Redo value of X, i.e. 9000;
 - Y ← Redo value of Y, i.e. 6000.
- (c) REDO (T_1, T_2) by moving forward:
 - X ← Redo value of X, i.e. 9000;
 - Y ← Redo value of Y, i.e. 6000;
 - Z ← Redo value of Z, i.e. 19000;

Advanced Recovery Techniques

The recovery processes for DBMS have gone through several changes. One of the basic objectives of any recovery technique is to minimise the time that is required to perform the recovery. However, in general, this reduction in time results in increased complexity of the recovery algorithms. One of the recent popular log-based recovery techniques is named ARIES. You may refer to further reading for more details on ARIES. In general, several aspects that increase the speed of the recovery process are:

- Transaction log sequence numbers for log records may be assigned to log entries. This will help in identifying the related database log pages.
- Instead of deletion the records from the physical database, record the deletion in the log.
- Keep track of pages that have been updated in memory but have not been written back to the physical database. Perform Redo operations for only such pages. Also, keep track of updated pages during checkpointing.

You may refer to further readings for more details on newer methods and algorithms of recovery.

11.3.4 Recovery with Concurrent Transactions

In general, a commercial database management system, such as a banking system, has many users. These users can perform multiple transactions. Therefore, a centralised database system executes many concurrent transactions. As discussed in Unit 10, these transactions may experience concurrency-related issues and, thus, are required to maintain serialisability. In general, these transactions share a large buffer area and log files. Thus, a recovery scheme that allows better control of disk buffers and large log files should be employed for such systems. One such technique is called checkpointing. This changes the extent to which REDO or UNDO operations are to be performed in a database system. The concept of the checkpoint has already been discussed in section 11.3.2. How checkpoints can help in the process of recovery is explained below:

When you use the checkpoint mechanism, you also add records of checkpoints in the log file. A checkpoint record is of the form: **<checkpoint TL>**. In this case, *TL* is the list of transactions, which were started, but not yet committed at the time when the checkpoint was created. Further, we assume that at the time of creating a checkpoint record, no transaction was allowed to proceed.

On the restart of the database system after failure, the following steps are performed for the database recovery:

- Create two lists: UNDOLIST and REDOLIST. Initialise both lists to a NULL.
- Scan the log file for every log record, starting from the end of the file and moving backwards, till you locate the first checkpoint record **<checkpoint TL>**. Perform the following actions, for each of the log records found in this step:
 - Is the log record **<T_i COMMIT>**?
 - If yes, then add *T_i* to REDOLIST.
 - Is the log record **<T_i START>**?
 - If yes, then Is *T_i* NOT IN REDOLIST?
 - Add *T_i* to UNDOLIST, as it has not yet been committed.
 - For every *T_i* in *TL*: Is *T_i* in NOT IN REDOLIST?
 - add *T_i* to UNDOLIST, as this transaction was active at the time of checkpoint and has not been committed yet.

This will make the UNDOLIST and REDOLIST of the transactions. Now, you can perform the UNDO operations followed by REDO operations using the log file, as given in section 11.3.3.

11.3.5 Buffer Management

As discussed in the previous sections, the database transactions are executed in the memory, which contains a copy of the physical database. These database buffers are written back to stable storage from time to time. In general, it is the memory management service of the operating system that manages the buffers of a database system. However, several database management schemes require their own buffer management policies and, hence, the buffer management system. Some of these strategies are:

Log Record Buffering

The recovery process requires database transactions to write logs in stable storage. This is a very time-consuming process, as for a single transaction several log records are to be written to stable storage. Therefore, several commercial database management systems perform the buffering of the log file itself, which means that log records are kept in the blocks of the main memory allocated for this purpose. The logs are then written to the stable storage once the buffer becomes full or when a transaction commits. This log file buffering helps in reducing disk accesses, which are very expensive in terms of time of operation.

Database recovery requires that log records should be stored in stable storage. Therefore, log records should be transferred from memory buffers to the table storage as per the following scheme, called Write-Ahead logging.

- The sequence of log records in the memory buffer should be maintained in stable storage.
- A transaction should be moved to COMMIT state only if the $\langle T_i \text{ commit} \rangle$ log record is written to stable storage.
- Prior to writing a database buffer to stable storage, related log records in the buffer should be moved to stable storage.

Database Buffering

The database updates are performed after moving database blocks on secondary storage to memory buffers. However, due to limited memory capacity, only a few database blocks can be kept in the memory buffers. Therefore, database buffer management consists of policies for deciding which blocks should be kept in database buffers and what blocks should be removed from the database buffers back to secondary storage. Removing a database block from the buffer requires that it is re-written to the secondary storage. In addition, the log is written to stable storage, as per the write-ahead logging.

11.3.6 Remote Backup Systems

Most of the techniques discussed above perform recovery in the context of a centralised system. However, present-day transaction processing systems require high availability. One of the ways of implementing a high availability system is to create a primary and a backup site for the transaction processing system. With fast, highly reliable networks, this backup site can be a remote backup site. This remote backup site may be very useful in case of disaster recovery. Some of the issues of the remote backup system include the following:

- The failure of the primary database site must be detected. It may be noted that this detection should not detect communication failure as a primary database failure. Thus, it may use a failsafe communication, which may have alternative communication links to the primary database site.
- The backup site should be capable enough to work as the primary database site at the time of failure of the primary site. In addition to recovery of ongoing transactions, once the primary site recovers it should get all the updates, which were performed while the primary site was down.

You may refer to the further readings for more details on this topic.



Check Your Progress 2

- 1) What is deferred database modification in recovery algorithms?

.....

-
- 2) How is log of the deferred database modification technique differ from the log of Immediate database modification?
-
-

- 3) Define buffer management and remote backup system in the context of recovery.
-
-

11.4 SECURITY IN COMMERCIAL DATABASES

You must realise that security is a journey, not the final destination. You cannot assume a product/technique is absolutely secure, as you may not be aware of fresh/new attacks on that product/technique. Many security vulnerabilities are not even published as attackers want to delay a fix, and manufacturers do not want negative publicity. There is an ongoing and unresolved discussion over whether highlighting security vulnerabilities in the public domain encourages or prevents further attacks.

The most secure database you can think of must be found in a most securely locked bank or nuclear-proof bunker, installed on a standalone computer without an Internet or network connection, and under guard for 24×7×365. However, that is not a likely scenario with which we would like to work. A database server maintains database services, which often contain security issues, and you should be realistic about possible threats. You must assume security failure at some point and never store truly sensitive data in a database that unauthorised users may easily infiltrate/access. A major point here is that most data loss occurs because of social exploits and not technical ones. Thus, the use of encryption algorithms for security may need to be looked into.

You will be able to develop effective database security if you realise that securing data is essential to the market reputation, profitability and business objectives. For example, personal information such as credit cards or bank account numbers are now commonly available in many databases; therefore, there are more opportunities for identity theft. As per an estimate, several identity theft cases are committed by employees who have access to large financial databases. Banks and companies that take credit card services externally must place greater emphasis on safeguarding and controlling access to this proprietary database information.

Securing the database is a fundamental tenet for any security personnel while developing his or her security plan. The database is a collection of useful data and can be treated as the most essential component of an organisation and its economic growth. Therefore, for any security effort, you must keep in mind that you need to provide the strongest level of control over the data of a database.

As is true for any other technology, the security of database management systems depends on many other systems. These primarily include the operating system, the applications that use the DBMS, services that interact with the DBMS, the web server that makes the application available to end users, etc. However, please note that most importantly, DBMS security depends on us, the users.

11.4.1 Common Database Security Failures

Database security is of paramount importance for an organisation, but many organisations do not take this fact into consideration till an eventual problem occurs. The common pitfalls that threaten database security are:

Weak User Account Settings: Many of the database user accounts do not contain the user settings that may be found in operating system environments. For example, the user accounts name and passwords, which are commonly known, are not disabled or modified to prevent access.

Insufficient Segregation of Duties: Several organisations have no established security administrator role. This results in database administrators (DBAs) performing both the functions of the administrator (for users' accounts), as well as the performance and operations expert. This may result in management inefficiencies.

Inadequate Audit Trails: The auditing capabilities of DBMS, since it requires keeping track of additional requirements, are often ignored for enhanced performance or disk space. Inadequate auditing results in reduced accountability. It also reduces the effectiveness of data history analysis. The audit trails record information about the actions taken on certain critical data. They log events directly associated with the data; thus, they are essential for monitoring the access and the activities on a database system.

Unused DBMS Security Features: The security of an individual application is usually independent of the security of the DBMS. Please note that security measures that are built into an application apply to users of the client software only. The DBMS itself and many other tools or utilities that can connect to the database directly through ODBC or any other protocol may bypass this application-level security completely. Thus, you must try to use security restrictions that are reliable, for instance, try using the security mechanisms that are defined within the DBMS.

11.4.2 Database Security Levels

Basically, database security can be broken down into the following levels:

- Server Security
- Database Connections
- Table Access Control

Server Security: Server security is the process of controlling access to the database server. This is the most important aspect of security and should be carefully planned. The basic idea here is “You cannot access what you do not see”. For security purposes, you should never let your database server be visible to the world. If a database server is supplying information to a web server, then it should be configured in such a manner that it is allowed connections from that web server only. Such a connection would require a trusted IP address.

Trusted IP Addresses: To connect to a server through a client machine, you would need to configure the server to allow access to only trusted IP addresses. You should know exactly who should be allowed to access your database server. For example, if the database server is the backend of a local application that is running on the internal network, then it should only talk to addresses from within the internal network.

Database Connections: With the ever-increasing number of Dynamic Applications, an application may allow immediate unauthenticated updates to some databases. If you are going to allow users to make updates to a database via a web page, please ensure that you validate all such updates. This will ensure that all updates are desirable and safe. For example, you may remove any possible SQL code from user-supplied input if a normal user is not allowed to input SQL code.

Table Access Control: Table access control is probably one of the most overlooked but one of the very strong forms of database security because of the difficulty in applying it. Using a table access control properly would require the collaboration of both the system administrator as well as the database developer. In practice, however, such “collaboration” is relatively difficult to find.

By now, we have defined some of the basic issues of database security, let us now consider specifics of server security from the point of view of network access of the system. Internet-based databases have been the most recent targets of security attacks.

All web-enabled applications listen to a number of ports. Cyber criminals often perform a simple “port scan” to look for ports that are open from the popular default ports used by database systems. How can we address this problem? We can address this problem “by default”, that is, we can change the default ports a database service would listen into. Thus, this is a very simple way to protect the DBMS from such criminals.

11.4.3 Relationship between Security and Integrity

Database security usually refers to the avoidance of unauthorised access and modification of data of the database, whereas database integrity refers to the avoidance of accidental loss of consistency of data. You may please note that data security deals not only with data modification but also access to the data, whereas data integrity, which is normally implemented with the help of constraints, essentially deals with data modifications. Thus, enforcement of data security, in a way, starts with data integrity. For example, any modification of data, whether unauthorised or authorised must ensure data integrity constraints. Thus, a very basic level of security may begin with data integrity but will require many more data controls. For example, SQL WRITE and UPDATE on specific data items or tables would be possible if it does not violate integrity constraints. Further, the data controls would allow only authorised WRITE and UPDATE on these data items.

11.4.4 Difference between Operating System and Database Security

Security within the operating system can be implemented at several levels ranging from passwords for access to the operating system to the isolation of concurrently executing processes within the operating system. However, there are a few differences between security measures taken at the operating system level compared to those of database system. These are:

- Database system protects more objects, as the data is persistent in nature. Also, database security is concerned with different levels of granularity such as files, tuples, attribute values or indexes. Operating system security is primarily concerned with the management and use of resources.
- Database system objects can be complex logical structures such as views, a number of which can map to the same physical data objects. Moreover, different architectural levels viz. internal, conceptual and external levels, have different security requirements. Thus, database security is concerned with the semantics – meaning of data, as well as with its physical representation. The operating system can provide security by not allowing any operation to be performed on the database unless the user is authorised for the operation concerned.

After this brief introduction to different aspects of database security, let us discuss one of the important levels of database security, access control, in the next section.

11.5 ACCESS CONTROL

All relational database management systems provide some sort of intrinsic security mechanisms that are designed to minimise security threats, as stated in the previous sections. These mechanisms range from the simple password protection offered in Microsoft Access to the complex user/role structure supported by advanced relational databases like Oracle, MySQL, Microsoft SQL Server, IBM Db2 etc. But can we define access control for all these DBMS using a single mechanism? SQL provides that interface for access control. Let us discuss the security mechanisms common to all databases using the Structured Query Language (SQL).

An excellent practice is to create individual user accounts for each database user. If

users are allowed to share accounts, then it becomes very difficult to fix individual responsibilities. Thus, it is important that we provide separate user accounts for separate users. Does this mechanism have any drawbacks? If the expected number of database users is small, then it is all right to give them individual usernames and passwords and all the database access privileges that they need to have on the database items.

However, consider a situation where there are a large number of users. Specification of access rights to all these users individually will take a long time. That is still manageable as it may be a one-time effort; however, the problem will be compounded if we need to change the access rights for a particular user. Such an activity would require huge maintenance costs. This cost can be minimised if we use a specific concept called “Roles”. A database may have hundreds of users, but their access rights may be categorised in specific roles, for example, teacher and student in a university database. Such roles would require the specification of access rights only once for each **role**. The users can then be assigned usernames, passwords, and specific roles. Thus, the maintenance of user accounts becomes easier as now we have limited roles to be maintained. You may study these mechanisms in the context of specific DBMS. A role can be defined using a set of data item/object authorisations. In the next sections, we define some of the authorisations in the context of SQL.

11.5.1 Authorisation of Data Items

Authorisation is a set of rules that can be used to determine which user has what type of access to which portion of the database. The following forms of authorisation are permitted on database items:

- 1) **READ:** it allows reading of data objects, but not modification, deletion, or insertion of a data object.
- 2) **INSERT:** allows insertion of new data, for example, insertion of a tuple in a relation, but it does not allow the modification of existing data.
- 3) **UPDATE:** allows modification of data, but not its deletion. However, data items like primary-key attributes may not be modified.
- 4) **DELETE:** allows deletion of data only.

A user may be assigned all, none, or a combination of these types of authorisations, which are broadly called access authorisations.

In addition to these manipulation operations, a user may be granted control operations such as:

- 1) **ADD:** allows adding new objects such as new relations.
- 2) **DROP:** allows the deletion of relations in a database.
- 3) **ALTER:** allows the addition of new attributes in a relation or deletion of existing attributes in a relation.
- 4) **Propagate Access Control:** this is an additional right that allows a user to propagate the access control or access right which s/he already has to some other user, for example, if user A has access right R over a relation S and if s/he has right to propagate access control, then s/he can propagate her/his access right R over relation S to another user B either fully or partially. In SQL, you can use **WITH GRANT OPTION** for this right.

The ultimate form of authority is given to the database administrator. S/he is the one who may authorise new users, restructure the database and so on. The process of authorisation involves supplying information only to the person who is authorised to access that information.

11.5.2 A basic model of Database Access Control

Models of database access control have grown out of earlier work on protection in

operating systems. Let us discuss one simple model with the help of the following example:

Example

Consider the relation:

Employee (Empno, Name, Address, Deptno, Salary, Assessment)

Assume there are two types of users: The personnel manager and the general user. What access rights may be granted to each user? One extreme possibility is to grant unconstrained access or to have limited access. One of the most influential protection models was developed by Lampson and extended by Graham and Denning. This model has 3 components:

- 1) A set of object entities to which access must be controlled.
- 2) A set of subject entities that request access to objects.
- 3) A set of access rules in the form of an authorisation matrix, as given in Figure 11 for the relation of the example.

Object Subject	Empno	Name	Address	Deptno	Salary	Assessment
Personnel Manager	Read	Read	All	All	All	All
General User	Read	Read	Read	Read	Not accessible	Not accessible

Figure 11: Authorisation Matrix for Employee relation.

As the above matrix shows, Personnel Manager and General User are the two subjects. Objects of the database are Empno, Name, Address, Deptno, Salary and Assessment. As per the access matrix, the personnel manager can perform any operation on the database of an employee except for updating the Empno and Name, which may be created once and can never be changed. The general user can only read the data but cannot update, delete or insert the data into the database. Also, the information about the salary and assessment of the employee is not accessible to the general user.

In summary, it can be said that the basic access matrix is the representation of basic access rules. These rules can be written using SQL statements, which are given in the next subsection.

11.5.3 SQL Support for Security and Recovery

You would need to create the users or roles before you grant them various permissions. The permissions then can be granted to a created user or role. This can be done with the use of the SQL GRANT statement.

The syntax of this statement is:

GRANT <permissions> [ON <table/view>] TO <user/role>
[WITH GRANT OPTION]

Let us define this statement line-by-line. The first line, GRANT <permissions>, allows you to specify the specific permissions on a table or a database view. These can be either relation-level data manipulation permissions (such as SELECT, INSERT, UPDATE and DELETE) or data definition permissions (such as CREATE TABLE, ALTER DATABASE and GRANT). More than one permission can be granted in a single GRANT statement, but data manipulation permissions and data definition permissions may not be combined in a single statement.

The second line, ON <table/view>, is used to specify the table or a view on which permissions are being given. This line is not needed if we are granting data definition permissions.

The third line specifies the user(s) or role(s) that is/are being granted permissions.

Finally, the fourth line, WITH GRANT OPTION, is optional. If this line is included *in the statement*, the user is also permitted to grant the same permissions that s/he has received to other users. Please note that the WITH GRANT OPTION cannot be specified when permissions are assigned to a *role*.

Let us look at a few examples of the use of this statement.

Example 1: Assume that you have recently hired a group of 25 data entry operators who will be adding and maintaining student records in a University database system. They need to be able to access information in the STUDENT table, modify this information and add new records to the table. However, they should not delete a record from the database.

Solution: First, you should create user accounts for each operator and then add them to a new role - DataEntry. Next, you will grant them the appropriate permissions, as given below:

```
GRANT SELECT, INSERT, UPDATE
ON STUDENT
TO DataEntry
```

And that is all that you need to do. The following example assigns data definition permissions to a role.

Example 2: You want to allow members of the DBA role to add new tables to your database. Furthermore, you want DBA to be able to grant permission to other users.

Solution: The SQL statement to do so is:

```
GRANT CREATE TABLE
TO DBA
WITH GRANT OPTION
```

Notice that we have included the WITH GRANT OPTION line to ensure that our DBAs can assign this permission to other users.

Let us now look at the commands for removing permissions from users.

Removing Permissions

Once we have granted permissions, it may be necessary to revoke them at a later date. SQL provides us with the REVOKE command to remove granted permissions. The following is the syntax of this command:

```
REVOKE [GRANT OPTION FOR] <permissions>
ON <table>
FROM <user/role>
```

Please notice that the syntax of this command is almost similar to that of the GRANT command. Please also note that the WITH GRANT OPTION is specified on the REVOKE command line and not at the end of the command as was the case in GRANT. As an example, let us imagine we want to revoke a previously granted permission to the user Usha, such that she is not able to remove records from the STUDENT database. The following SQL command will be able to do so:

```
REVOKE DELETE
ON STUDENT
FROM Usha
```

The access control mechanisms supported by the SQL is a good starting point, but you must look into the DBMS documentation to locate the enhanced security measures supported by your system. You will find that many DBMS support more advanced access control mechanisms, such as granting permissions on specific attributes.

SQL does not have very specific commands for recovery but, it allows explicit COMMIT, ROLLBACK and other related commands.

11.6 AUDIT TRAILS IN DATABASES

One of the key issues to consider while procuring a database security solution is making sure you have a secure audit trail. An audit trail tracks and reports activities around confidential data. Many companies have not realised the potential amount of risk associated with sensitive information within databases unless they run an internal audit which details who has access to sensitive data and have assessed it. Consider the situation in which a DBA who has complete control of database information may conduct a security breach with respect to business details and financial information. This will cause tremendous loss to the company. In such a situation database audit helps in locating the source of the problem. The database audit process involves a review of log files to find and examine all reads and writes to database items during a specific time period to ascertain mischief, if any. A banking database is one such database which contains very critical data and should have the security feature of auditing. An audit trail is a log that is used for the purpose of security auditing.

Database auditing is one of the essential requirements for security, especially for companies in possession of critical data. Such companies should define their auditing strategy based on their knowledge of the application or database activity. Auditing need not be of the type “all or nothing”. One must do intelligent auditing to save time and reduce performance concerns. This also limits the volume of logs and also causes more critical security events to be highlighted.

More often than not, it is the insiders who make database intrusions as they often have network authorisation, knowledge of database access codes and the idea about the value of data they want to exploit. Sometimes, despite having all the access rights and policies in place, database files may be directly accessible (either on the server or from backup media) to such users. Most database applications store information in ‘form text’ that is completely unprotected and viewable.

As huge amounts are at stake, incidents of security breaches will increase and continue to be widespread. For example, a large global investment bank conducted an audit of its proprietary banking data. It was revealed that more than ten DBAs had unrestricted access to their key sensitive databases, and over a hundred employees had administrative access to the operating systems. The security policy that was in place was that proprietary information in the database should be denied to employees who did not require access to such information to perform their duties. Further, the bank’s database internal audit also reported that the backup data (which is taken once every day) also caused concern as backup media could get stolen. Thus, the risk to the database was high and real and the bank needed to protect its data.

However, a word of caution, while considering ways to protect sensitive database information, please ensure that the privacy protection process should not prevent authorised personnel from obtaining the right data at the right time.

Credit card information is the single most common financially traded information that is desired by database attackers. The positive news is that database misuse or unauthorised access can be prevented with currently available database security products and audit procedures.



Check Your Progress 3

- 1) On what systems does the security of a Database Management System depend?

.....
.....

- 2) Write the syntax for granting permission to alter the database.

.....
.....

- 3) Write the syntax for 'Revoke Statement' that revokes the grant option.

.....
.....

- 4) What is the main difference between data security and data integrity?

.....
.....

11.7 SUMMARY

In this unit, we have discussed the recovery of the data contained in a database system after failure. Database recovery techniques are methods of making the database fault tolerant. The aim of the recovery scheme is to allow database operations to be resumed after a failure with no loss of information and at an economically justifiable cost. The basic technique to implement database recovery is to use data redundancy in the form of logs and archival copies of the database. Checkpoint helps the process of recovery.

Security and integrity concepts are crucial. The DBMS security mechanism restricts users to only those pieces of data that are required for the functions they perform. Security mechanisms restrict the type of actions that these users can perform on the data that is accessible to them. The data must be protected from accidental or intentional (malicious) corruption or destruction.

Security constraints guard against accidental or malicious tampering with data; integrity constraints ensure that any properly authorised access, alteration, deletion, or insertion of the data in the database does not change the consistency and validity of the data. Database integrity involves the correctness of data, and this correctness has to be preserved in the presence of concurrent operations. The unit also discussed the use of audit trails.

11.8 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) Recovery is needed to take care of the failures that may be due to software, hardware and external causes. The aim of the recovery scheme is to allow database operations to be resumed after a failure with the minimum loss of information and at an economically justifiable cost. One of the common techniques is log-based recovery. The transaction is the basic unit of recovery.
- 2) All recovery processes require redundancy. Log-based recovery process records the consistent state of the database and all the modifications made by a transaction into a log on the stable storage. In case of any failure, the stable log and the database states are used to create a consistent database state.
- 3) A checkpoint is a point when all the database updates and logs are written to stable storage. A checkpoint ensures that not all the transactions need to be REDONE or UNDONE. Thus, it helps in faster recovery from failure. The checkpoint helps in recovery, as in case of a failure all the committed transactions prior to the checkpoint are NOT to be redone. Only non-committed transactions at the time of checkpoint or transactions that started after the checkpoint are required to be REDONE or UNDONE based on the log.

Check Your Progress 2

- 1) The deferred database modification recovery algorithm postpones, as far as possible, the writing of updates into the physical database. Rather, the modifications are recorded in the log file, which is written into stable storage. In case of a failure, the log can be used to REDO the committed transactions. In this process UNDO operation is not performed.
- 2) The log of deferred database modification technique just stores the REDO information. UNDO information is not required as updates are performed in the main memory buffers only, while the immediate modification scheme maintains both the UNDO and REDO information in the log
- 3) Buffer management is important from the point of view of recovery, as it may determine the time taken in recovery. It is also used for implementing different recovery algorithms.
Remote backup is very useful in the case of disaster recovery. It may also make the database available even if one site of the database fails.

Check Your Progress 3

- 1) The database system security may depend on the security of the Operating system, including the network security; and the application that uses the database and services that interact with the web server.
- 2) GRANT ALTER DATABASE TO UserName
- 3) REVOKE GRANT OPTION
FOR <permissions> ON <table> FROM <user/role>
- 4) Data security is the protection of information that is maintained in the database against unauthorised access, modification or destruction. Data integrity is the mechanism that is applied to ensure that data in the database is correct and consistent.